📘 Level 1 Notes — Java Basics & Coding Challenges

Save as:

`Java_Level_1_Basics_Coding_Challenges.md`

Overview

Level 1 focuses on becoming comfortable with **basic Java syntax and problem solving**.

The goal isn't to learn complicated algorithms. Instead, the goal is to become comfortable writing Java without constantly checking syntax.

By the end of this level, you should be comfortable with:

- Variables
- Primitive types
- Strings
- Arrays
- Loops
- Conditions
- Methods
- `StringBuilder`
- `HashMap`
- `HashSet`
- Basic Collections
- Basic problem solving

---

1. Reverse a String

Challenge

Reverse:

```
hello
```

into:

```
olleh
```

Solution

```
private static String reverseString(String str) {

    StringBuilder result = new StringBuilder();

    for (int i = str.length() - 1; i >= 0; i--) {
        result.append(str.charAt(i));
    }

    return result.toString();
}
```

Concepts

- `StringBuilder`
- `for` loop
- `length()`
- `charAt()`
- `append()`
- `toString()`

---

2. Character Frequency

Challenge

Count how many times each character appears.

```
banana
```

Result:

```
b → 1
a → 3
n → 2
```

Solution

Solution:

```
private static Map<Character, Integer> countCharFrequency(String str) {

    Map<Character, Integer> frequency = new HashMap<>();

    for (int i = 0; i < str.length(); i++) {

        Character c = str.charAt(i);

        if (frequency.containsKey(c)) {
            frequency.put(c, frequency.get(c) + 1);
        } else {
            frequency.put(c, 1);
        }
    }

    return frequency;
}
```

Important Map Syntax

```
map.containsKey(key)
```

Checks whether the key exists.

```
map.get(key)
```

Gets the value.

```
map.put(key, value)
```

Adds or updates a value.

Modern shorthand:

```
map.put(
    c,
    map.getOrDefault(c, 0) + 1
);
```

---

3. Find Duplicate Characters

Challenge

Find duplicate characters:

```
programming
```

Expected:

```
r
g
m
```

Solution

```
public static Set<Character> findDuplicates(String str) {

    Set<Character> seen = new HashSet<>();
    Set<Character> duplicates = new HashSet<>();

    for (int i = 0; i < str.length(); i++) {

        Character c = str.charAt(i);

        if (seen.contains(c)) {
            duplicates.add(c);
        } else {
            seen.add(c);
        }
    }

    return duplicates;
}
```

Concepts

```
HashSet
contains()
add()
```

---

4. Check Palindrome

Challenge

Check whether:

```
madam
```

is a palindrome.

Solution

```
public static boolean isPalindrome(String str) {

    String reversed = reverseString(str);

    return str.equals(reversed);
}
```

Important

Use:

```
str.equals(other)
```

for String comparison.

Do **not** use:

```
str == other
```

for String content comparison.

---

5. Find Minimum and Maximum

Challenge

Find the minimum and maximum value in an array.

```
Integer[] numbers = {
    -1, -7, -3, -4, -4, -6, -5
};
```

Solution

Solution

```
public static String findMinAndMax(Integer[] numbers) {

    int max = Integer.MIN_VALUE;
    int min = Integer.MAX_VALUE;

    for (int num : numbers) {

        if (num > max) {
            max = num;
        }

        if (num < min) {
            min = num;
        }
    }

    return "Max value " + max +
           " and Min value " + min;
}
```

Important

Using:

```
Integer.MIN_VALUE
Integer.MAX_VALUE
```

makes the solution work with negative numbers too.

---

6. Remove Duplicates

Challenge

Remove duplicate numbers.

```
10, 20, 30, 20, 10
```

Solution

```
public static Set<Integer> removeDuplicates(
        Integer[] numbers) {
```

```
    return new HashSet<>(
        Arrays.asList(numbers)
    );
}
```

Important

`HashSet` automatically guarantees uniqueness.

---

7. Find Second Largest Number

Challenge

Find the second-largest **distinct** number.

Solution

```
public static Integer findSecondLargestNumber(
        Integer[] numbers) {

    int firstHighest = Integer.MIN_VALUE;
    int secondHighest = Integer.MIN_VALUE;

    for (int num : numbers) {

        if (num > firstHighest) {

            secondHighest = firstHighest;
            firstHighest = num;

        } else if (
                num > secondHighest &&
                num < firstHighest
        ) {

            secondHighest = num;
        }
    }

    return secondHighest;
}
```

Important pattern

```
new highest
```

```
    ↓
old highest → second highest
new number → highest
```

---

8. Count Words

Challenge

Count words in:

```
Hi I am a full stack developer
```

Solution

```
public static Integer countWordsInASentence(
        String str) {

    return str.trim()
              .split("\\s+")
              .length;
}
```

Important

```
"\\s+"
```

means one or more whitespace characters.

It handles multiple spaces:

```
Hello    World
```

---

9. FizzBuzz

Challenge

For numbers from `1` to `n`:

```
3   → Fizz
5   → Buzz
15  → FizzBuzz
```

Solution

```
public static ArrayList<String> fizzBuzz(int n) {

    ArrayList<String> result =
            new ArrayList<>();

    for (int i = 1; i <= n; i++) {

        if (i % 3 == 0 && i % 5 == 0) {

            result.add("FizzBuzz");

        } else if (i % 3 == 0) {

            result.add("Fizz");

        } else if (i % 5 == 0) {

            result.add("Buzz");

        } else {

            result.add(String.valueOf(i));
        }
    }

    return result;
}
```

Important syntax

```
i % 3
```

gets the remainder.

```
String.valueOf(i)
```

converts the number to a String.

---

10. Check Anagrams

10. Check Anagrams

Challenge

Check:

```
listen
silent
```

Expected:

```
true
```

Solution

```
public static boolean checkAnagrams(
        String s1,
        String s2) {

    if (s1.length() != s2.length()) {
        return false;
    }

    Map<Character, Integer> frequency =
            new HashMap<>();

    for (int i = 0; i < s1.length(); i++) {

        Character c = s1.charAt(i);

        frequency.put(
                c,
                frequency.getOrDefault(c, 0) + 1
        );
    }

    for (int i = 0; i < s2.length(); i++) {

        Character c = s2.charAt(i);

        if (!frequency.containsKey(c)) {
            return false;
        }

        if (frequency.get(c) > 1) {

            frequency.put(
                    c,
```

```
                    frequency.get(c) - 1
            );

        } else {

            frequency.remove(c);
        }
    }

    return frequency.isEmpty();
}
```

Important Map pattern

Instead of:

```
if (map.containsKey(c)) {
    map.put(c, map.get(c) + 1);
} else {
    map.put(c, 1);
}
```

you can use:

```
map.put(
    c,
    map.getOrDefault(c, 0) + 1
);
```

This is a very useful Java pattern.

Level 1 — What You Should Remember

```
String
 ├── length()
 ├── charAt()
 ├── equals()
 └── StringBuilder

Array
 ├── length
 └── enhanced for-loop

HashMap
 ├── containsKey()
```

```
  ├── get()
  ├── put()
  ├── remove()
  └── getOrDefault()

HashSet
  ├── add()
  ├── contains()
  └── uniqueness

Loops
  ├── for
  ├── enhanced for
  └── if / else

Methods
  ├── parameters
  ├── return type
  └── return
```